AI ACADEMY · LEVEL 2 · GRADE 7 · WEEK 4 · STUDENT LESSON

Trace Tables

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Transparent Book Recommender

THE CONNECTED WEEK

Record the state. Follow the range. Explain the first mismatch.

Trace accumulator updates, expose repeated resets and compare initialization with zero iterations.

Keep one habit: Predict range values → record before/after state → distinguish print events from updates.

MAIN LESSON · SECTION 1

What changes on each iteration?

> Trace initialization, repeated updates and the point where output occurs.

Start with state rather than the final answer

A trace records state before and after an instruction. In a loop, keep the current turn, the old total, the new total and each print event separate.

Recall the earlier trace and addition update

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Starting with total = 2, what does total = total + 3 store?

Your learning goals

- I can **trace variable** values before and after each **loop iteration**.
- I can use the excluded stop value in range correctly.
- I can explain **initialization** and test zero one and several **iteration**s.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Your learning target

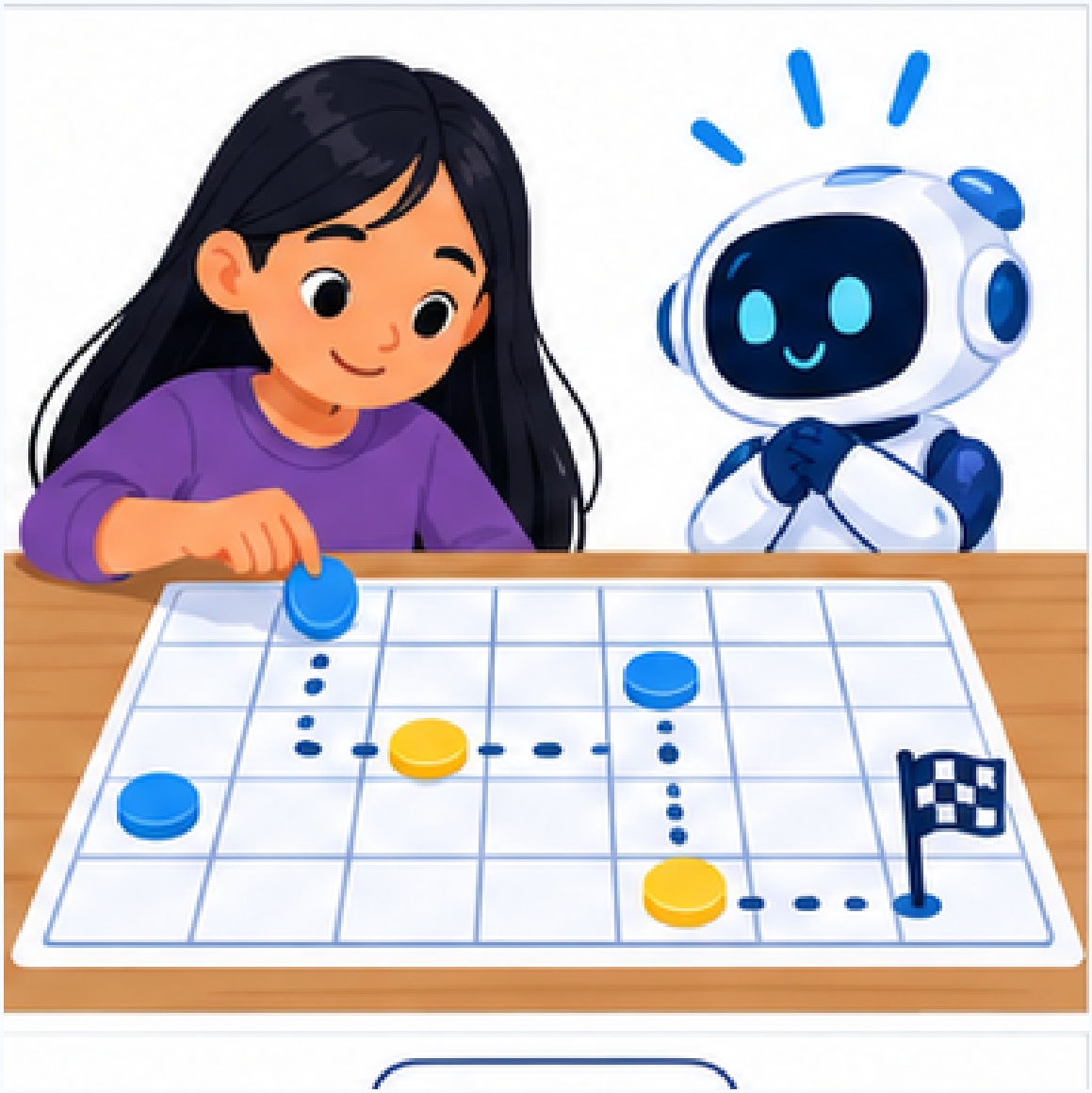
Remember before reading

Without opening the old lesson, explain one key idea from Branching Logic. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Precision in Algorithms. Give an example and a mistake that the idea helps you avoid.

Week 4: **Trace Table**s

A **trace table** records changing **state** and reveals reasoning errors.
</details>



THINKER MISSION

Trace variables through a score-based game helper.

Thinker name and date:

Write or draw your reasoning and evidence.

Readiness for the connected explanation

This week's end goal

By the end, I can make a **trace table** for a for-**loop**, including zero **iteration**s.

MAIN LESSON · SECTION 2

Meet the numbered-counter loop

> Use integer ranges with the default positive step of one.

State the repeated-addition task

Imagine adding numbered counters: first 1, then 2, then 3. A **loop** can repeat the same addition with a different number each time. This week you will predict each change before running a short Python script.

Tracing repeated calculations prepares you to inspect the data **process**ing used in larger AI applications.

Read the included start and excluded stop

In for turn in range(1, 4), Python supplies 1, 2 and 3 in order. The start is included and the stop is excluded. The indented body runs once for each supplied value. The colon and four-space indentation mark that body.

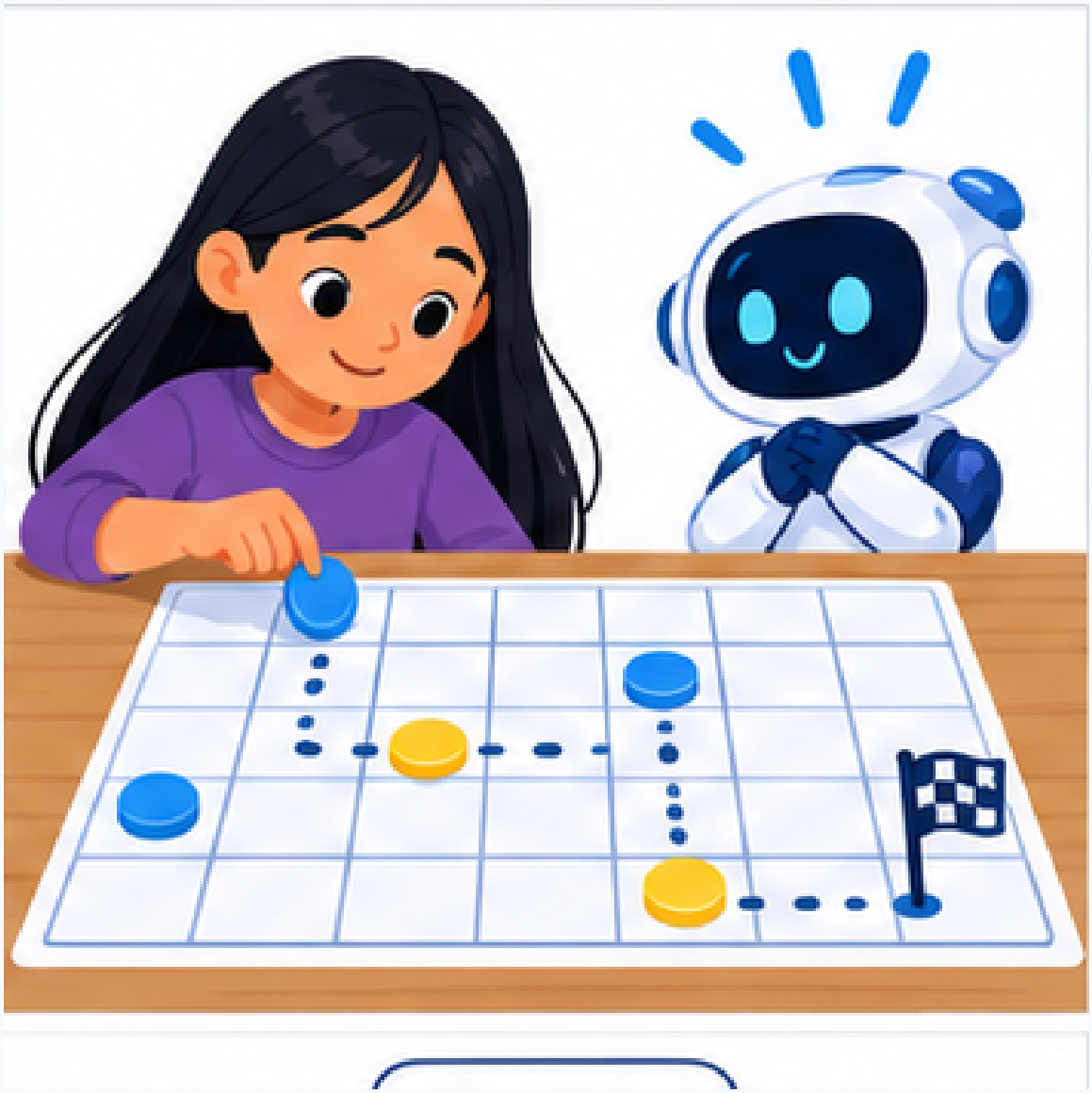
This lesson uses integer range values with the default positive step of 1.

▼ **Earlier wording, alternatives and picture transcripts**

PICTURE STORY

Notice → Predict → Explain

Begin with visible **evidence**. Do not explain more than the picture and information support.



MYSTERY

The learners must use **trace table**s to solve a problem. What information do they have? What decision or calculation do they make? What would count as **evidence** that it worked?

Three **observation**s (what you can point to):

> **Write or draw your reasoning and evidence.**

My **prediction** and the clue supporting it:

> **Write or draw your reasoning and evidence.**

A closer explanation

MAIN LESSON · SECTION 3

Run the complete accumulator script

> The main loop adds the changing turn value, then prints once after the loop.

The complete saved Python script

```
total = 0
for turn in range(1, 4):
    total = total + turn
print(total)
```

Read total before and after each update

State after each addition

TURN	TOTAL BEFORE	TOTAL AFTER
1	0	1
2	1	3
3	3	6

Initialize total to 0 before the loop. The unindented print runs once afterward.

Explain initialization, exhaustion and final printing

- The **output** is 6. Before the first **iteration**, total is 0. Each assignment reads the previous total and adds the current turn. After turn 3, the range is exhausted, so execution continues after the **loop**.

  The table records **state**; it does not create extra instructions.

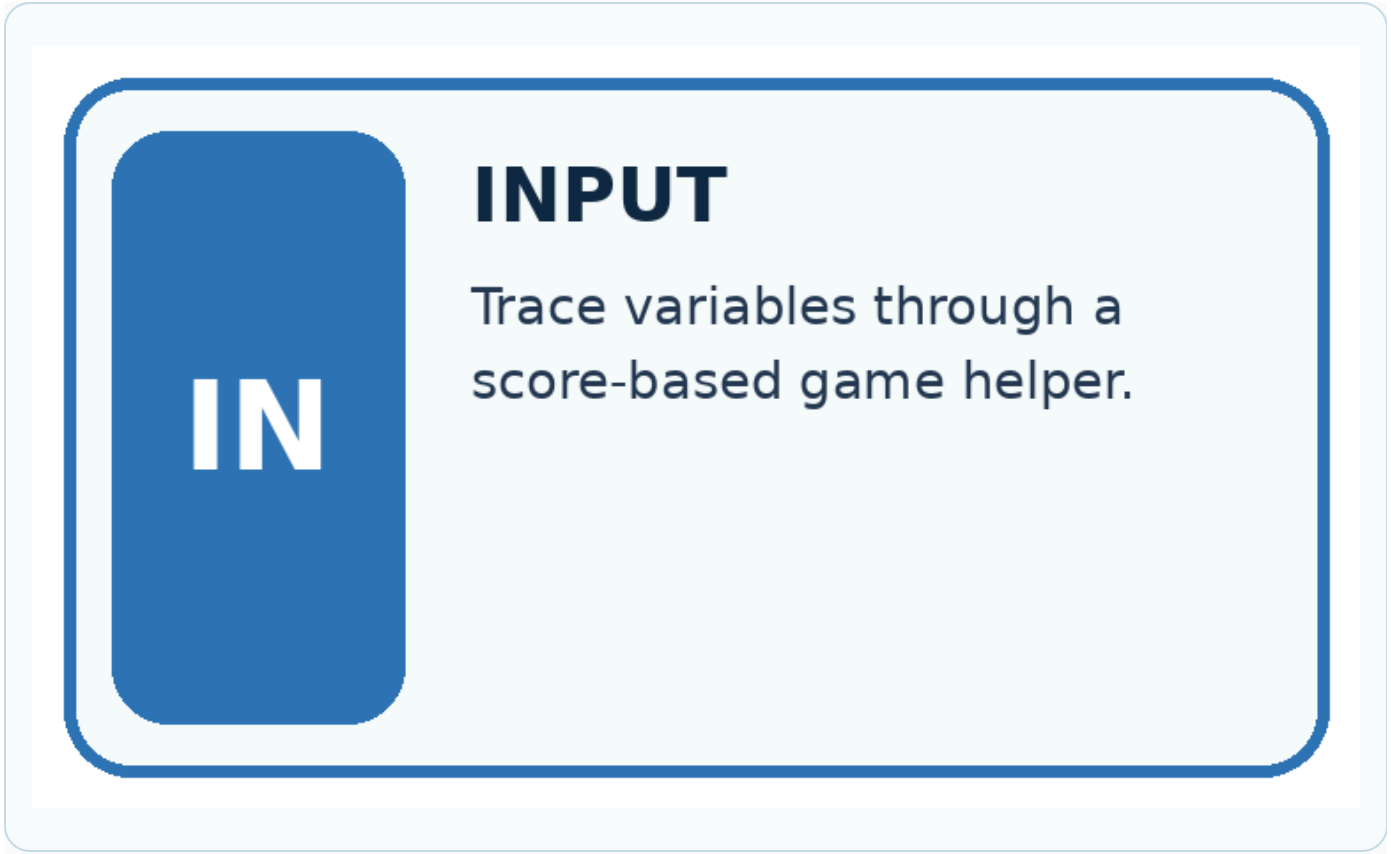
- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ Earlier wording, alternatives and picture transcripts

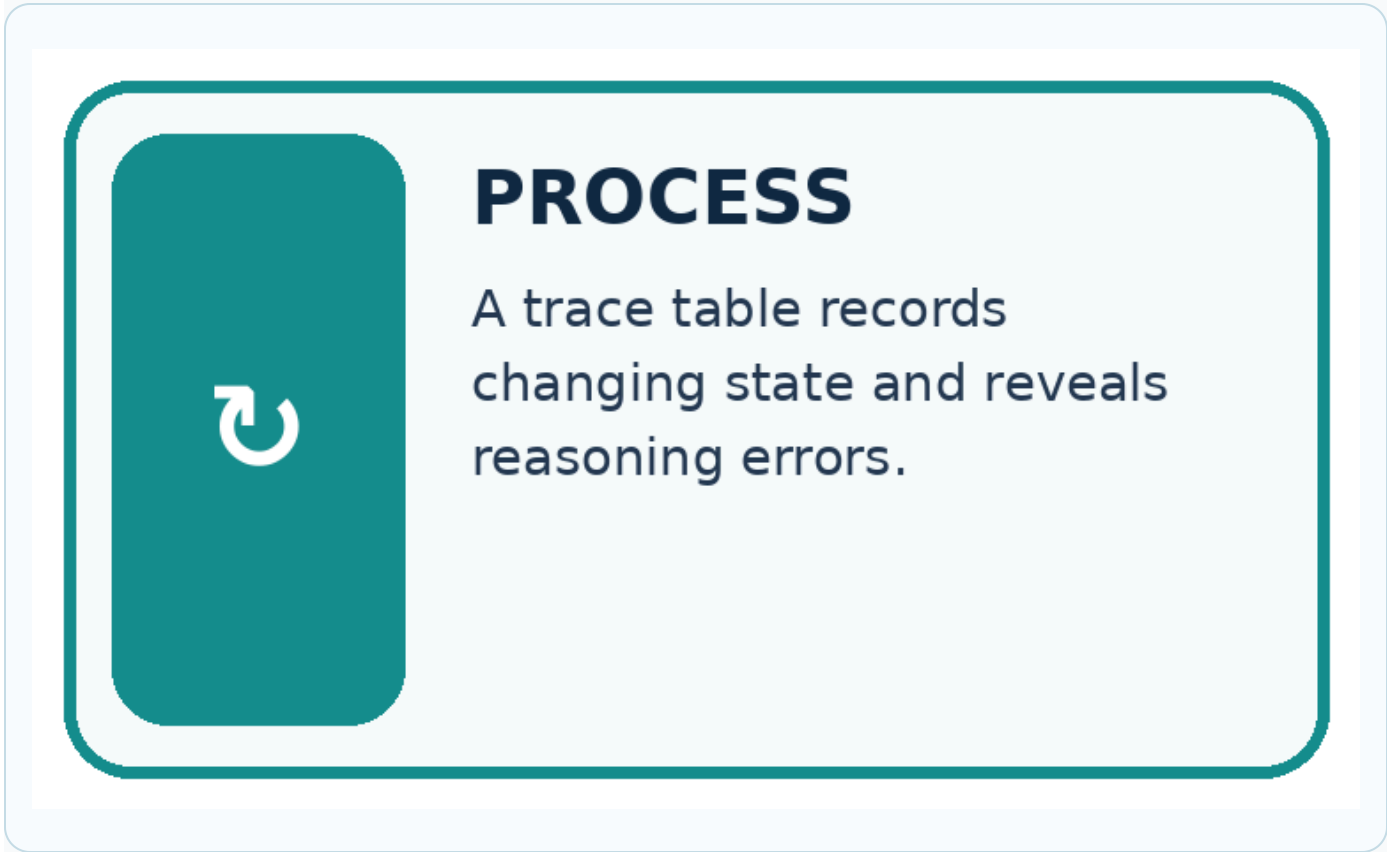
VISUAL **MECHANISM**

Input → **Process** → **Output**

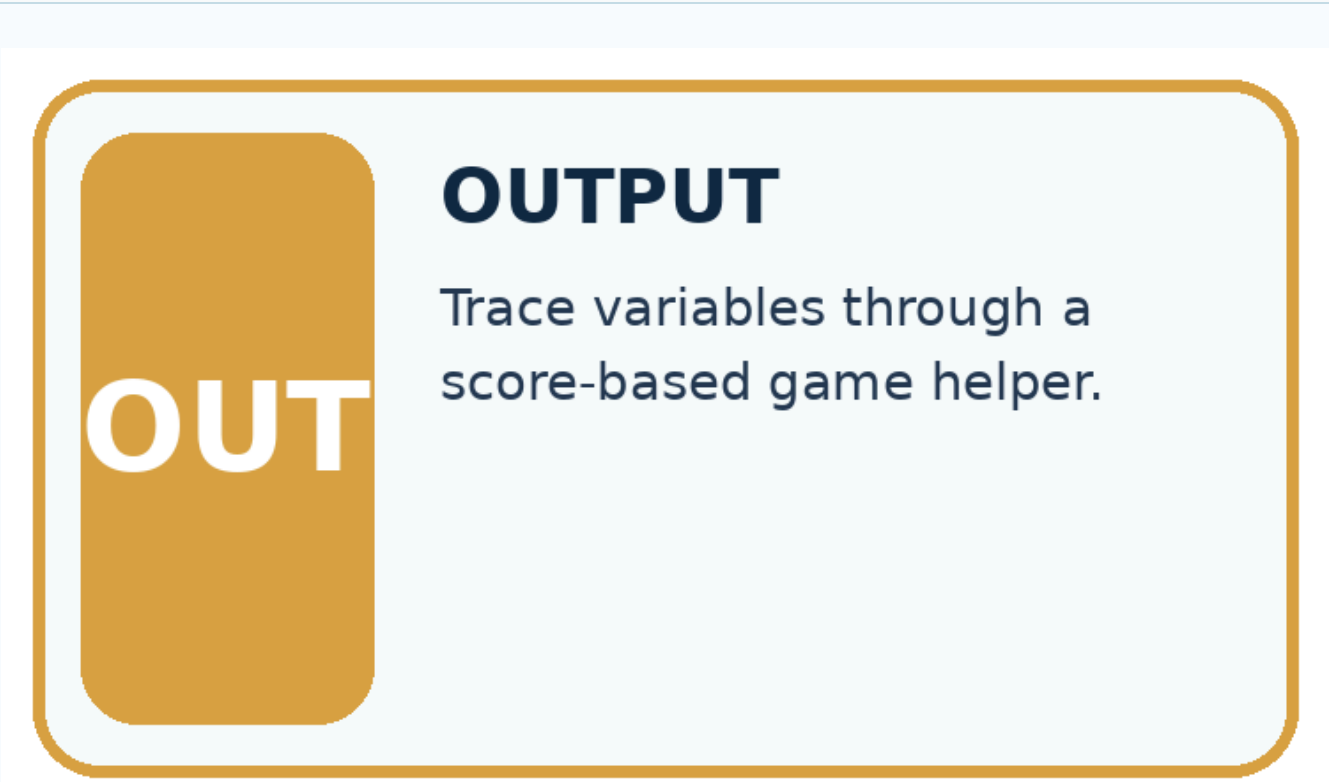
Trace the information. At each stage, ask what changed and how you could test it.


IN
INPUT
Trace variables through a score-based game helper.

↓


PROCESS
A trace table records changing state and reveals reasoning errors.

↓



Which stage could fail even when the other two are correct? Explain:

Write or draw your reasoning and evidence.

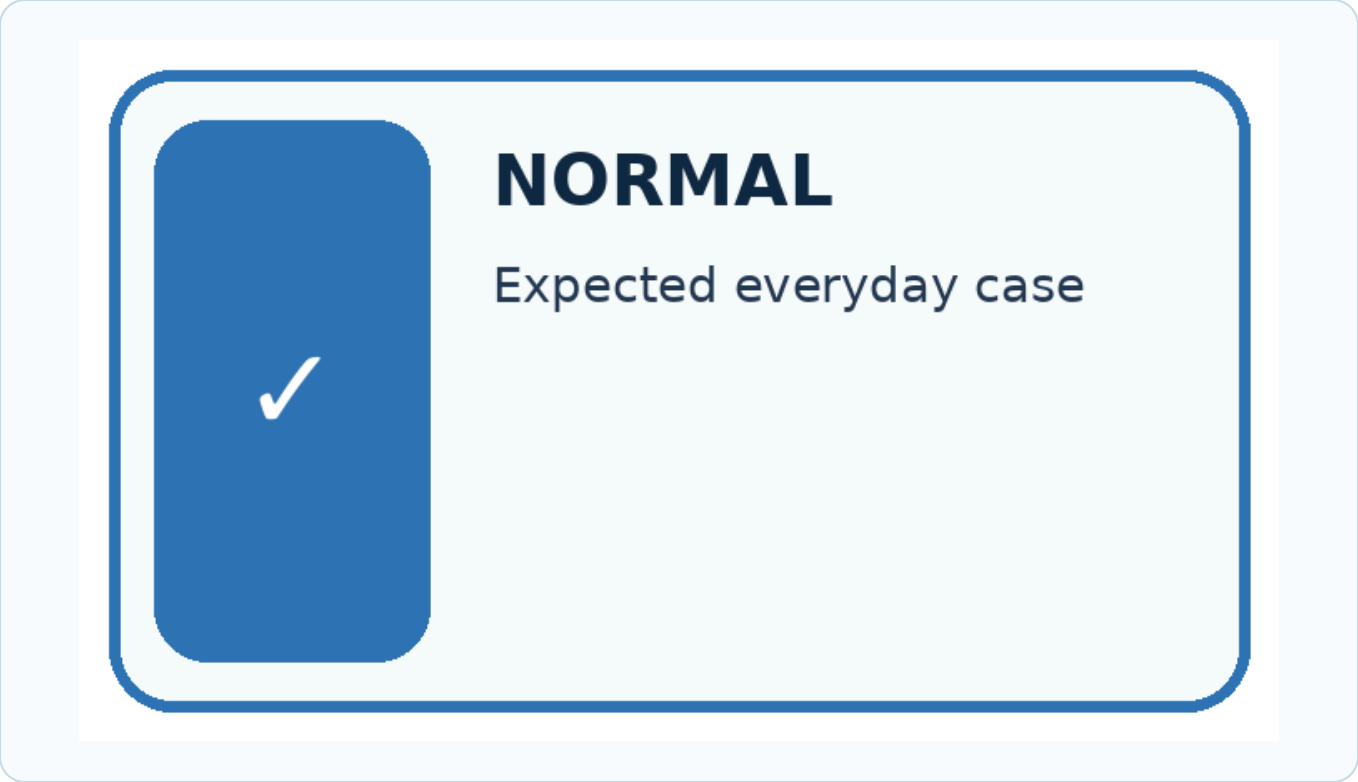
A closer explanation

Connect the picture and the explanation

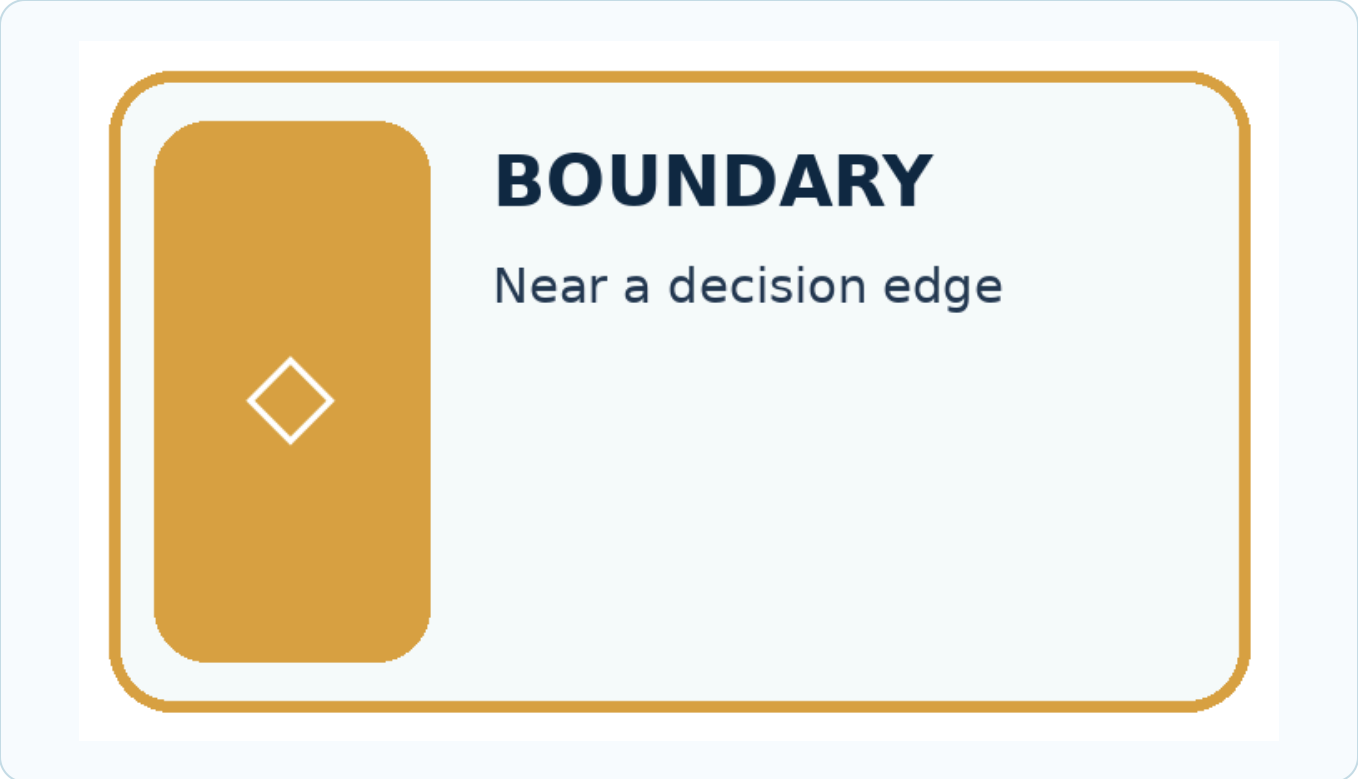
Compare routine, zero-iteration, missing and transfer traces

> One iteration can hide an accumulator reset that several iterations reveal.

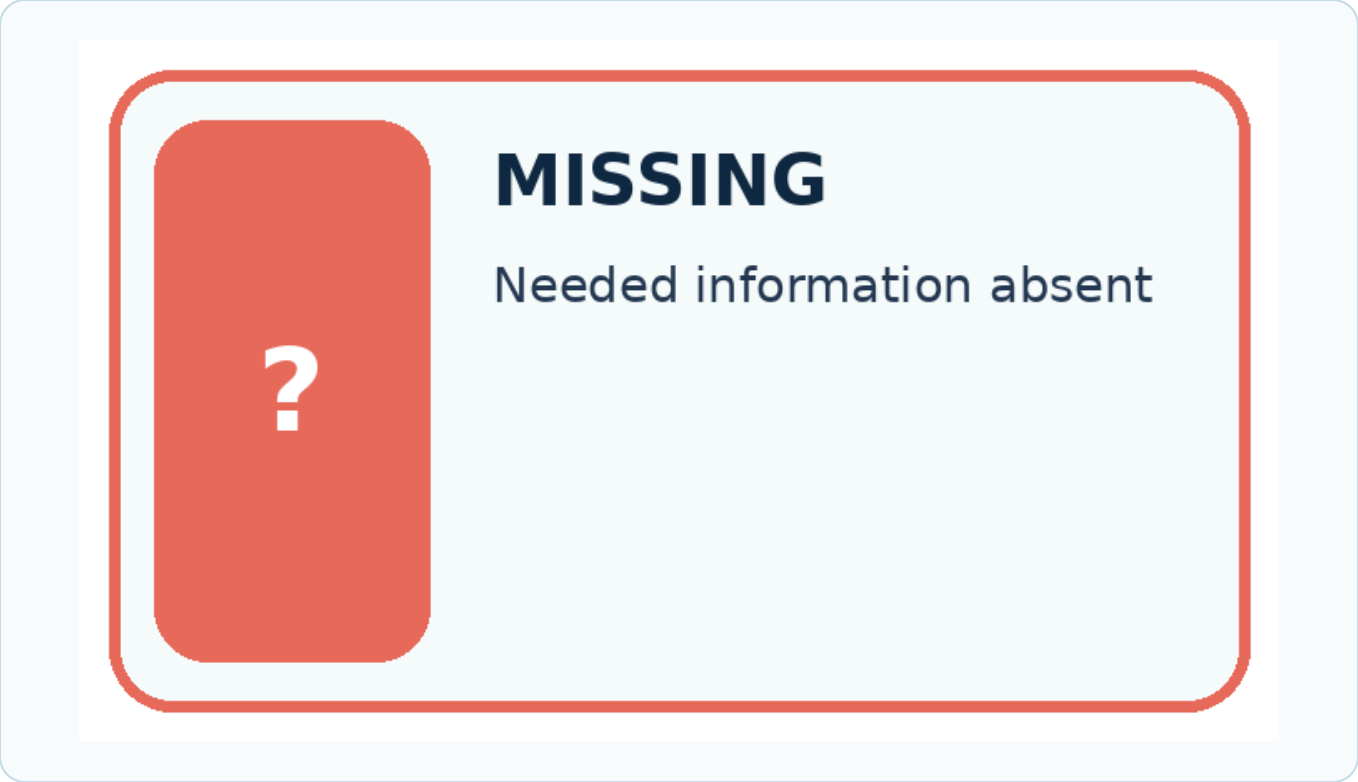
Four pictures with a purpose



Normal case: predict the result and deciding evidence.



Boundary case: identify the exact limit.



Missing information: name what must be clarified.



Transfer case: explain which procedure still applies.

Choose a revealing case

- Which case is most likely to reveal a hidden assumption? Why?

> **Write or draw your reasoning and evidence.**

Predict the supplied changed case

- Change: Use range(0) instead of range(4).

▼ Check after predicting

Prediction: The **loop** body never runs and the printed total is 1.

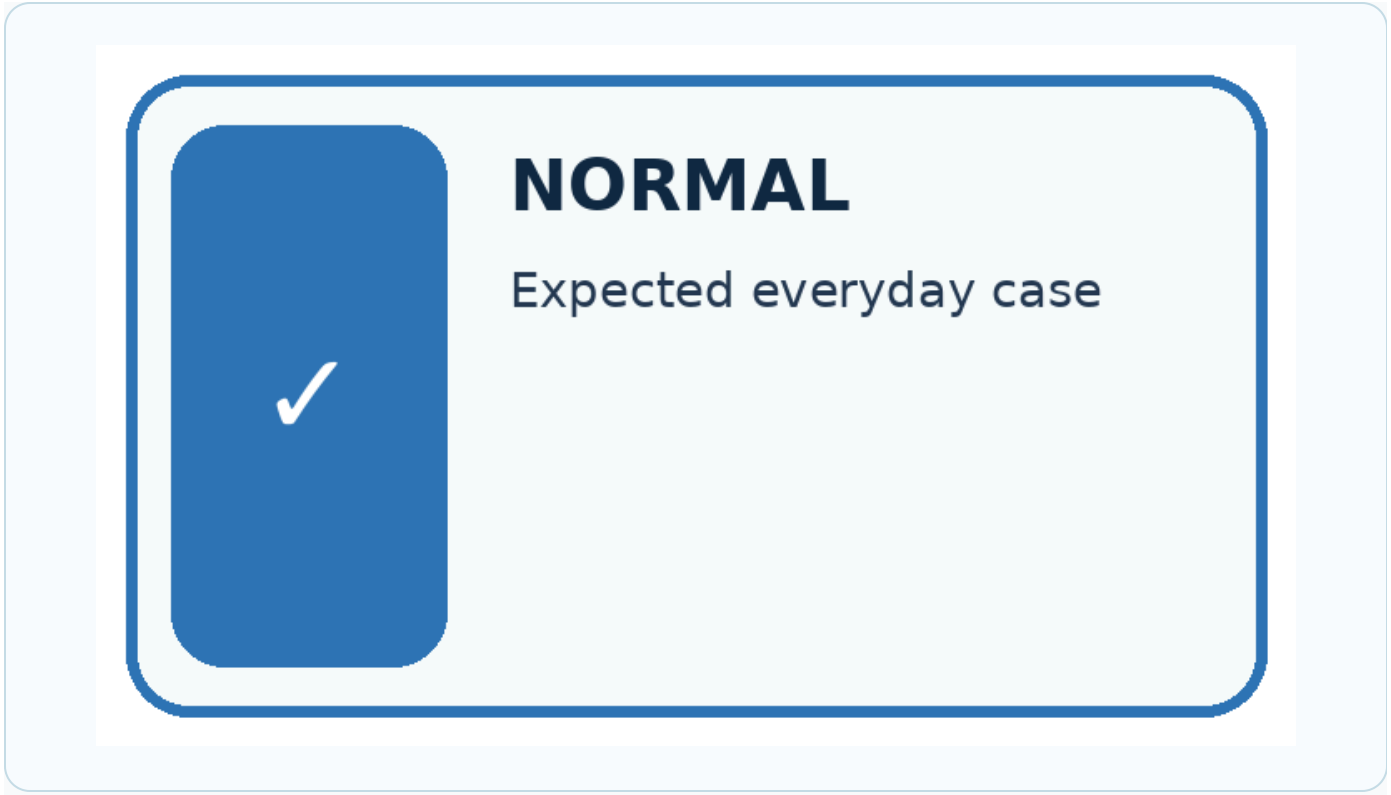
Reason: The iterable contains no turn values; the initial assignment still happened.

▼ Earlier wording, alternatives and picture transcripts

CASE GALLERY

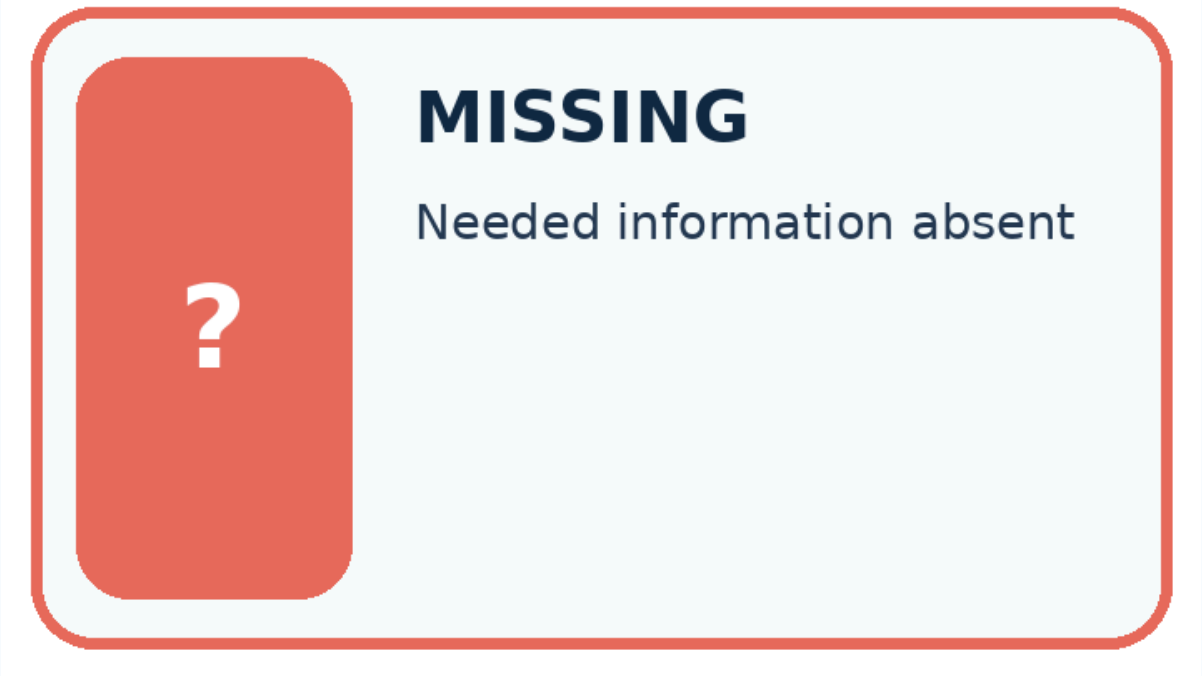
Four Tests, Four Kinds of **Evidence**

A system is not understood until it survives more than one comfortable example.





NORMAL CASE: predict and name **evidence**. BOUNDARY CASE: predict and name **evidence**.





MISSING INFORMATION: predict and name **evidence**. TRANSFER CASE: predict and name **evidence**.

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Compare range values and the separate token-jar trace

> The jar adds a fixed three each round; the main counter program adds turn.

Trace zero, one and several range values

Range expression	Values supplied	Number of iterations
range(1, 4)	1, 2, 3	3
range(1, 2)	1	1
range(1, 1)	No values	0
range(4)	0, 1, 2, 3	4

With one argument, range starts at zero. range(4) supplies four values but none is 4. If the worked script uses range(1, 1), the body never runs and the initialized total remains 0.

Do not rely on a **variable** first assigned inside a **loop**: a zero-**iteration** run might never assign it.

Predict the shortened main range

Replace range(1, 4) with range(1, 3) in the worked script. Predict the supplied values and final total. What stop value is needed to include 3?

Inspect the main loop/range diagram



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the separate jar's initial value and update

A fictional reading-token jar starts with one token and gains three per round.

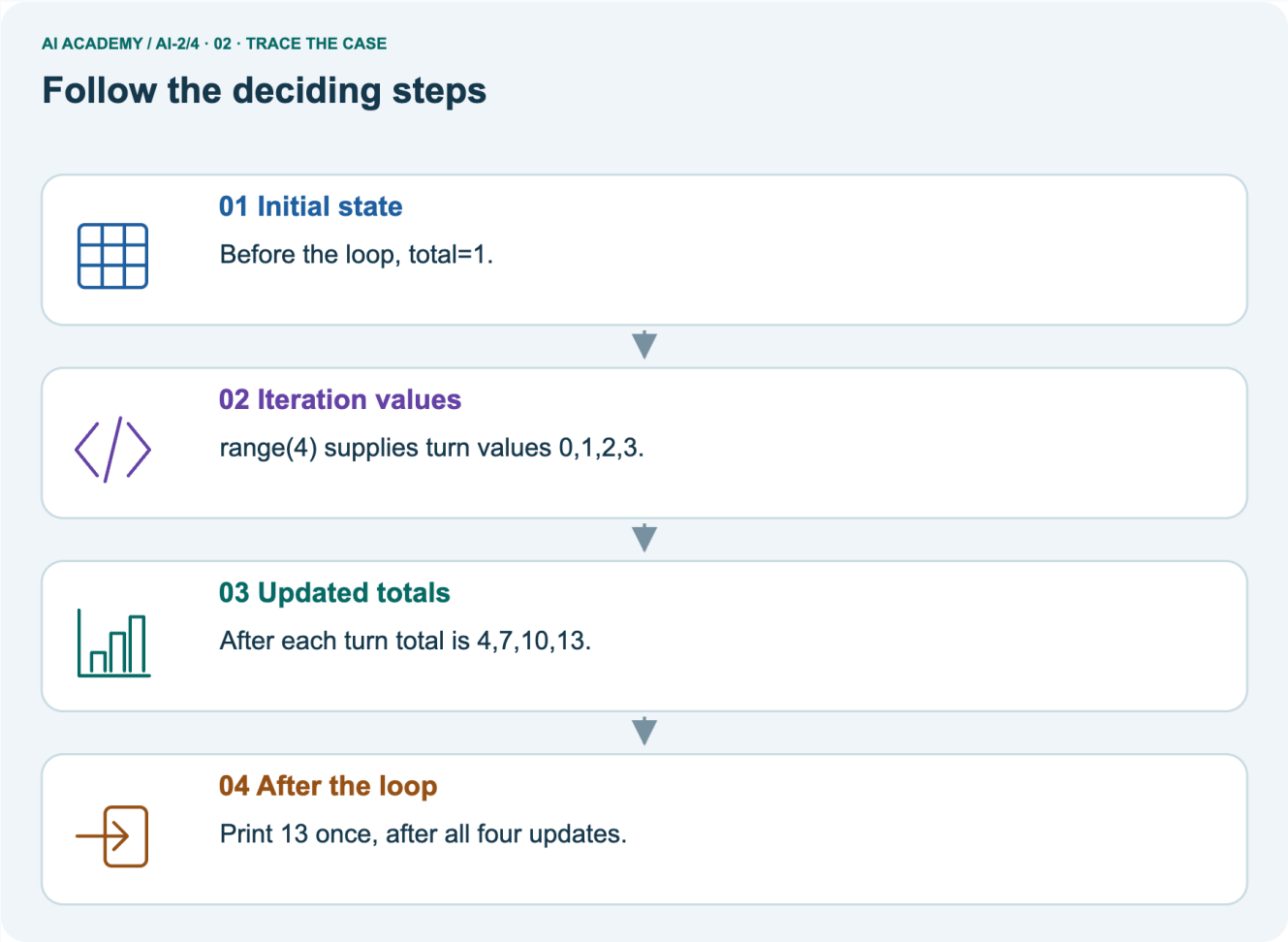
Python: total=1; for turn in range(4): total=total+3; after the **loop** print(total).

The update is inside the **loop**; print is outside.

```
total = 1
for turn in range(4):
```

```
    total = total + 3
print(total)
```

Trace four fixed-size updates



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Initial state	Before the loop, total=1.
2. Iteration values	range(4) supplies turn values 0,1,2,3.
3. Updated totals	After each turn total is 4,7,10,13.
4. After the loop	Print 13 once, after all four updates.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

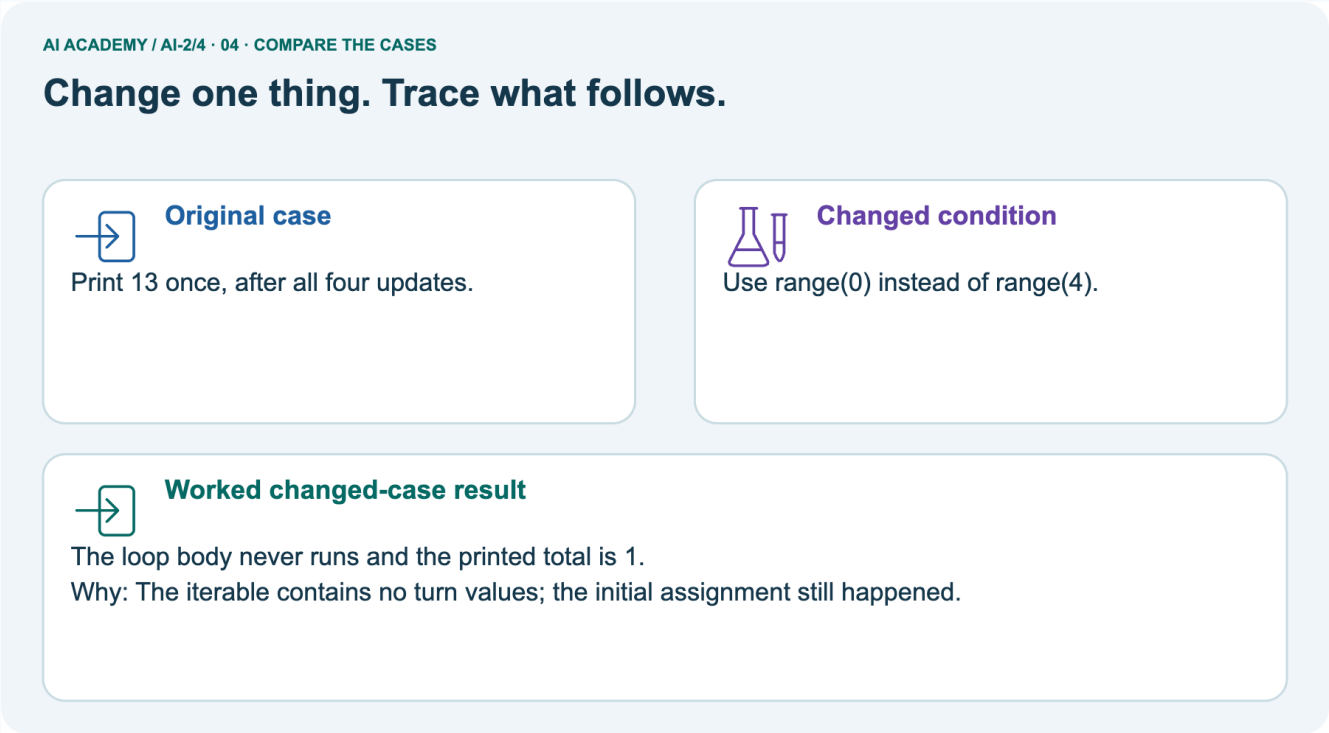
The turn value labels an **iteration**; it is not the number added. The fixed update adds three each time. Four **iteration**s take the total from 1 to 13.

Keeping print outside the **loop** produces one final display rather than four intermediate displays. Initialise total before the **loop** so a zero-**iteration** run still has a defined value.

Predict zero iterations from the saved initial state

- Before opening the zero-round jar picture, use range(0) with total initially 1. Predict the final printed value. Explain why this differs from the main numbered-counter program, whose initial total is zero.

▼ Check the zero-round jar result



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain print placement and the scope of the finite loop

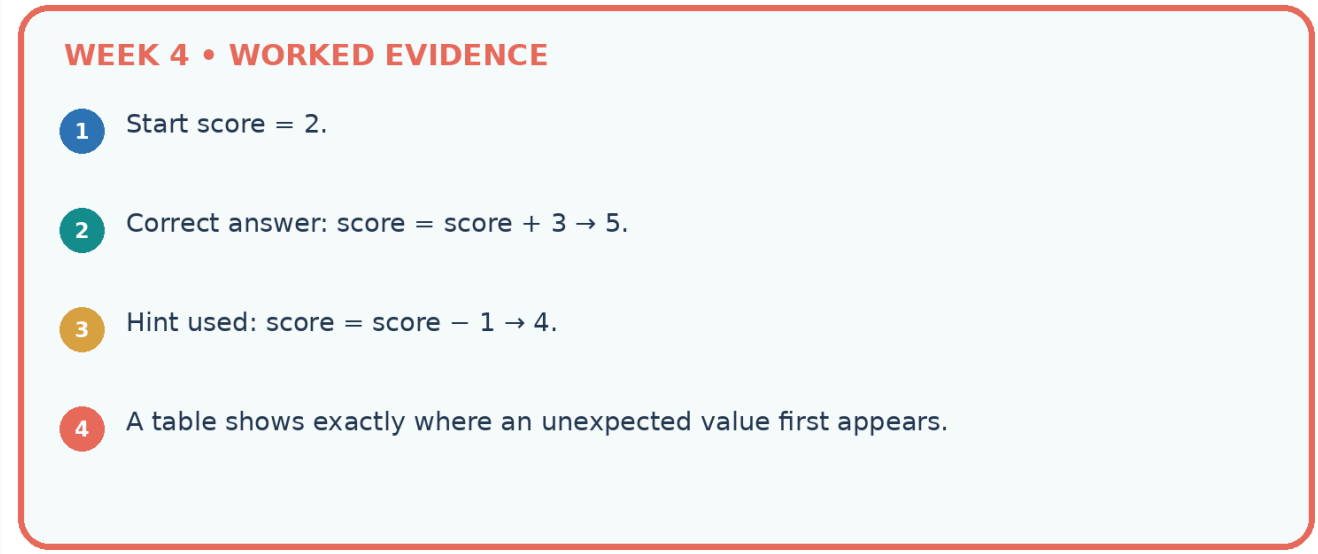
- Read the labelled picture: Move print inside the original **loop**. What sequence is displayed?
- This finite **loop** does not establish behavior of a while-**loop** with a different update or stopping condition.

▼ Earlier wording, alternatives and picture transcripts

WORKED **EVIDENCE**

Reason Step by Step

Predict first. Then check each step. A correct answer without visible reasoning is incomplete **evidence**.



1. Start score = 2.

2. Correct answer: score = score + 3 → 5.

3. Hint used: score = score – 1 → 4.

4. A table shows exactly where an unexpected value first appears.

Recalculate, re**trace**, or re**state** the reasoning in your own representation:

> **Write or draw your reasoning and evidence.**

CHECK Did the **evidence** answer the exact question? Did the total, path, or source record stay consistent?

A closer explanation

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Trace Tables

The facts we will use

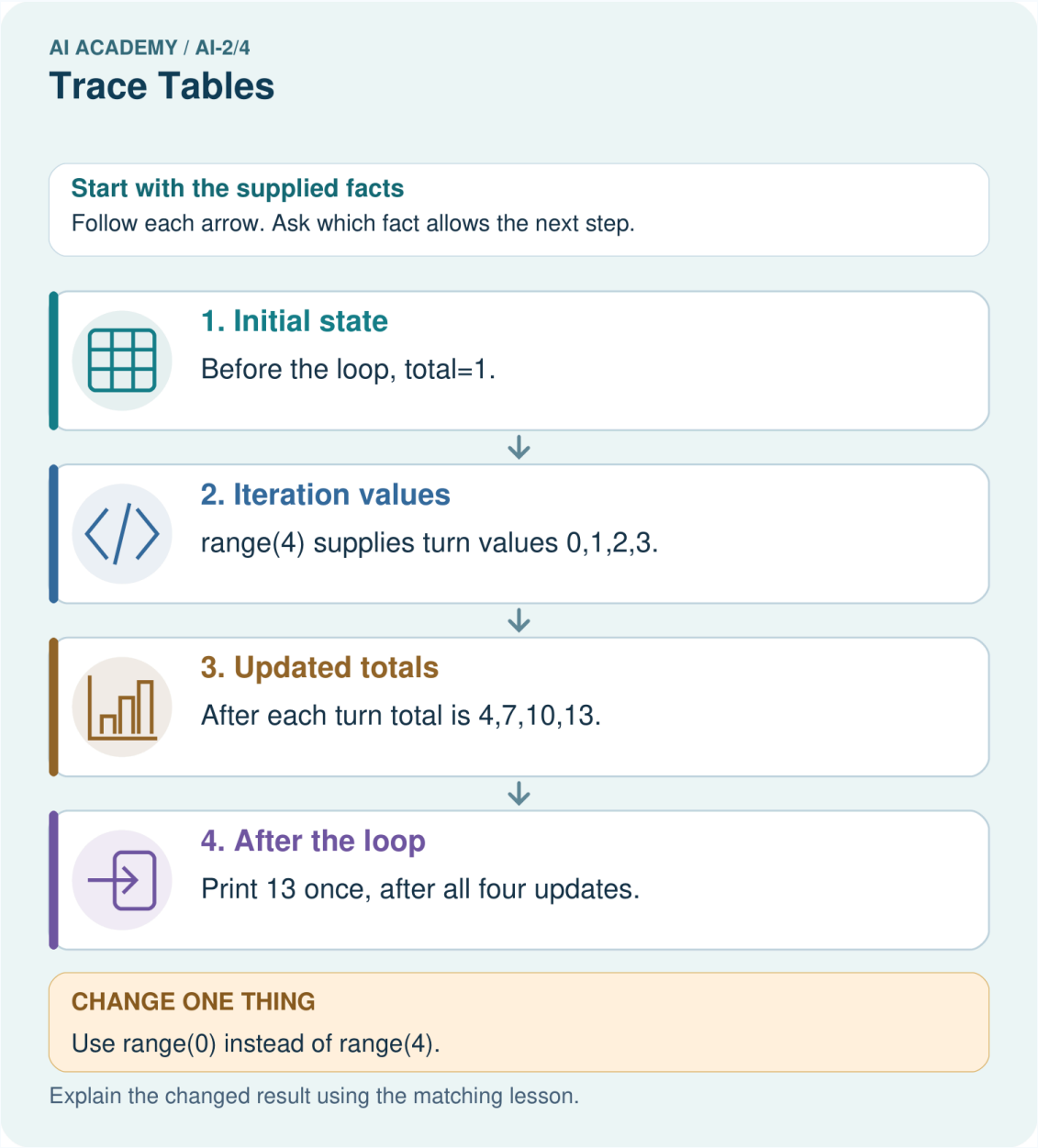
AI ACADEMY / AI-2/4

Loop totals after each iteration

	turn	total after update
First	0	4
Second	1	7
Third	2	10
Fourth	3	13

Read a cell using both its row and column labels.

Follow the labelled picture with the matching explanation. Open the vector diagram



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Change one thing: Use range(0) instead of range(4). Predict the result before checking the explanation. What remains the same?

Keep traces that can reveal the first divergence

> Test zero, one and several updates instead of relying on a single matching run.

Write expectations and retain actual observations

Write the expected range values and **trace table** before running. Keep predicted and observed **output** separate, then explain the first row where they differ.

For a **loop** that should add a fixed amount repeatedly, zero **iteration**s should preserve the initial amount. One **iteration** checks one update; several check accumulation.

Challenge a one-iteration reset test

> **A learner checks the reset bug using only range(1, 2) and sees the expected answer 1. Explain why this does not establish correctness, and give a reveling test with its expected correct output.**

▼ Earlier wording, alternatives and picture transcripts

ERROR DETECTIVE

Compare → Diagnose → Repair

Do not patch the answer. Find the earliest unsupported assumption, wrong step, or weak **evidence**.

!

COMPARE • DIAGNOSE

Weak idea: A trace table records only the final answer. Strong move: locate evidence, change one cause, and retest.

Evidence record	Expected	Actual	Likely cause / next test
normal case	works		
boundary case	stated rule		
missing input	safe response		
fresh transfer	generalizes		

Repair one cause. Why is this repair better than changing several things?

Write or draw your reasoning and evidence.

A closer explanation

Practise: repair a reset without moving initialization

> An indented reset can discard earlier work on every iteration.

Run the faulty accumulator script

```
total = 0
for turn in range(1, 4):
    total = 0
    total = total + turn
print(total)
```

Locate the discarded additions and targeted repair

This faulty version prints 3. The extra reset discards earlier additions on every **iteration**. Remove the indented reset while keeping **initialization** before the **loop**.

Then test zero, one and several **iteration**s: the one-**iteration** case alone cannot reveal this reset bug.

Compare printing inside and outside the loop

Moving print(total) into the correct **loop** produces three lines, 1 then 3 then 6. Moving it out produces only the final 6. **Trace** both values and **output** events.

A **loop** follows written instructions; repeating them does not make the program learn.

Trace the changed main range in W2

Use workbook W2 for the connected example “**Loop**s and **trace table**s”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your **trace** to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Repair the included-stop claim

- Use the worked case for **Loop**s and **trace table**s. Proposed explanation: “The stop value is included”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Return to the jar’s intermediate outputs

Move print inside the original **loop**. What sequence is displayed?

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Earlier wording, alternatives and picture transcripts

LOGIC STUDIO

Trace Before You Run

Block code, pseudocode, tables, and diagrams are different ways to expose the same reasoning.

TRACE THE LOGIC

Predict the result. Trace every condition or calculation. Compare expected with actual. Explain the first difference.

SET **evidence**_record PREDICT expected_result RUN one normal case and one boundary case COMPARE actual_result with expected_result IF different: locate first cause; change one factor; RETEST EXPLAIN result, limitation, and next safe step

Trace one **input**. Which line changes the **state** or chooses a path?

Write or draw your reasoning and evidence.

A closer explanation

Practise with less help: Trace before running

Check your reasoning

Explain the picture

My explanation using a pictured step and the given facts

MAIN LESSON · SECTION 9

Investigate changing state and hidden reset errors

> Use fully supplied score steps and loop variants.

Prepare the paper score trace

Paper score trace: start score=2, add 3, if the current score>4 subtract 1, then double the current score. Use columns step, condition/event, old score and new score; record a condition when it is checked, not against the initial value. Keep this straight-line/branch task separate from the loop task that follows.

Predict, run, diagnose and retest the loop

1. Code trace: use the complete numbered-counter script. Before each fresh run predict the supplied turn values, total before/after each iteration and final output for range(1,4), range(1,2) and range(1,1). Then run the supplied reset-bug version on the same ranges, retain the actual results, remove only the extra indented reset and rerun. Compare a separately labelled version printing after every addition. Record output events as well as state; label paper-only results traces, not executions.

Keep the evidence table

Test / input · Prediction · Actual · Evidence / calculation · Change or keep?

> **Test / input:** normal
>
> **Prediction:** _____
>
> **Actual:** _____
>
> **Evidence / calculation:** _____
>
> **Change or keep?:** _____

Test / input: boundary

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: missing

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: fresh transfer

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Explain the most revealing test

- What did the hardest test teach you that the normal test did not?

- Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only **trace**d the procedure.

Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

EVIDENCE LAB

Predict → Test → Record → Explain

Keep expected and actual results separate. Surprises are useful data when recorded honestly.



Use columns for step, event, old score, new score, and branch. **Trace** once by hand, predict the next row, then compare.

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: build the different repeated counter addition

> Use its initial five and fixed addition of two before feedback.

The independent rounds contract

A box starts with 5 counters. Each round adds 2 counters. Use rounds = 3, an initialized total and for turn in range(rounds). Print the final total once after the **loop**.

Trace the totals and predict fresh runs with rounds 0, 1 and 3. Assume rounds is a nonnegative integer.

Keep your first attempt

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice **evidence** together.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Build a repeated addition independently

Week 4 Independent application

Close the worked explanation. Apply the idea to this different question without copying.

Set score to 2. Add 3, then double the result. Record the score after each operation. What changes if you double before adding?

My answer, with a **trace**, calculation or supporting **evidence**

Write or draw your reasoning and evidence.

Create and test

Change one relevant detail. Predict the result before testing. Include a boundary or missing-information case when it helps reveal a limit.

My changed **input** and expected result

Write or draw your reasoning and evidence.

Connect to your project

Trace each book's genre, length, difficulty, and total score before sorting.

The **evidence** I will keep

Write or draw your reasoning and evidence.

Transfer practice from the original programme

Start score=1; add 4; if score>3 subtract 2; double score. Record score after every operation.
A student evaluates the condition using the initial value. Locate the divergence.

Explain the mechanism, evidence and one limit.

Use feedback to improve

MAIN LESSON · SECTION 11

Carry the trace method into the recommender portfolio

> Show each update and its declared rule before making a ranking claim.

Scope of this week's contribution

The Transparent Book Recommender will need inspectable eligibility and score updates for each fictional book. This week practise the trace method on the supplied small examples. Record the current item and old/new values when project rules are provided; do not invent a scoring formula or claim that repetition trains a model.

The checkpoint and system map

THIS WEEK'S CHECKPOINT **Trace** each book's genre, length, difficulty, and total score before sorting.

SYSTEM MAP Reader need → allowed **input**s → transparent rule or score → ranked candidates → explanation → test record → **human decision**

Trace recommender **loop**s with current item, eligibility, score and current best so each update can be explained.

Record the change and human choice

- What will you add, change, or verify in the project this week?

- What should remain under **human control**?

Write or draw your reasoning and evidence.

Write or draw your reasoning and evidence.

Keep privacy and evidence limits visible

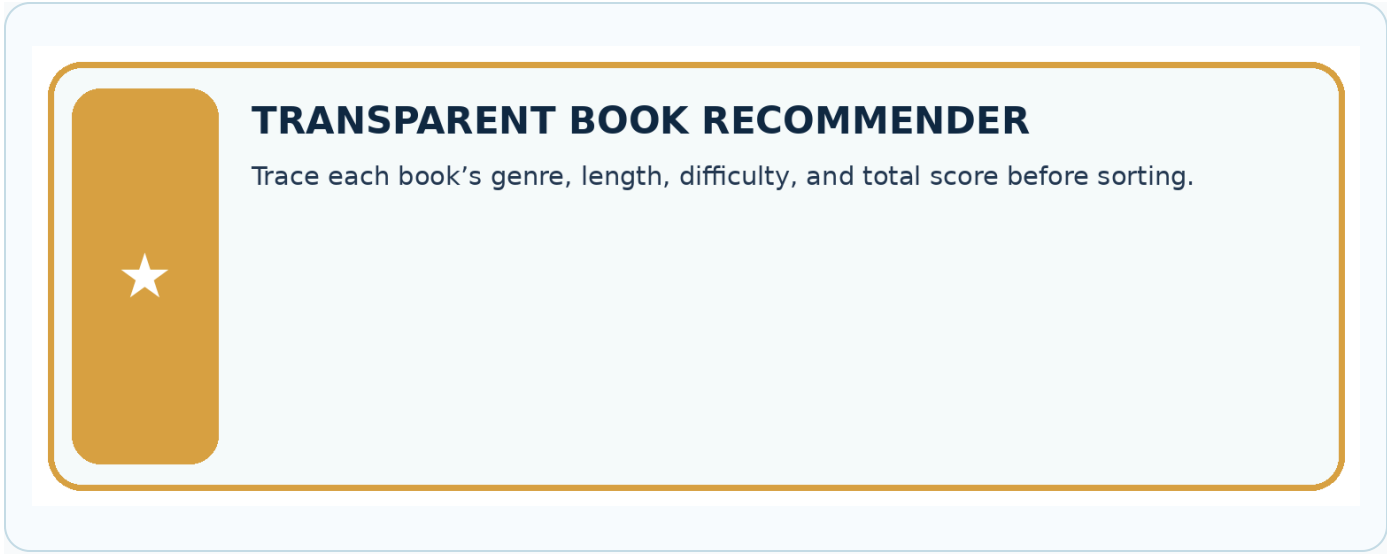
RESPONSIBLE DESIGN Collect only necessary information. Show uncertainty and limits.
Never confuse a ranking with a fact about a person.

▼ **Earlier wording, alternatives and picture transcripts**

CUMULATIVE PROJECT

Transparent Book Recommender

Every week adds one inspectable piece. The system must show how it reached a result.



Evidence for your project

MAIN LESSON · SECTION 16

Finish: explain the initial state and each update

> Close your notes and distinguish a turn value from the amount added.

Your exit explanation

- Close your notes. Explain this goal in your own words: **Trace variable** values before and after each **loop iteration**

My deciding step, evidence and one remaining question

Choose the next revealing iteration count

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Your quick word guide**

REFERENCE · SECTION 5

Your quick word guide

> Use the words to explain a particular step, record or limit.

Core vocabulary

Word	My own example	Not this / boundary
variable	A named reference to a value that can be used or changed in a program.	
state	The current values and information that determine what a process does next.	
trace	A record of the successive states, decisions or information transformations in a process.	
iteration	One repeat of a process, often followed by inspection and revision.	

Terms for the connected explanation

Word	Meaning
Loop	A block of instructions that can run repeatedly
Accumulator	A variable that keeps a running result

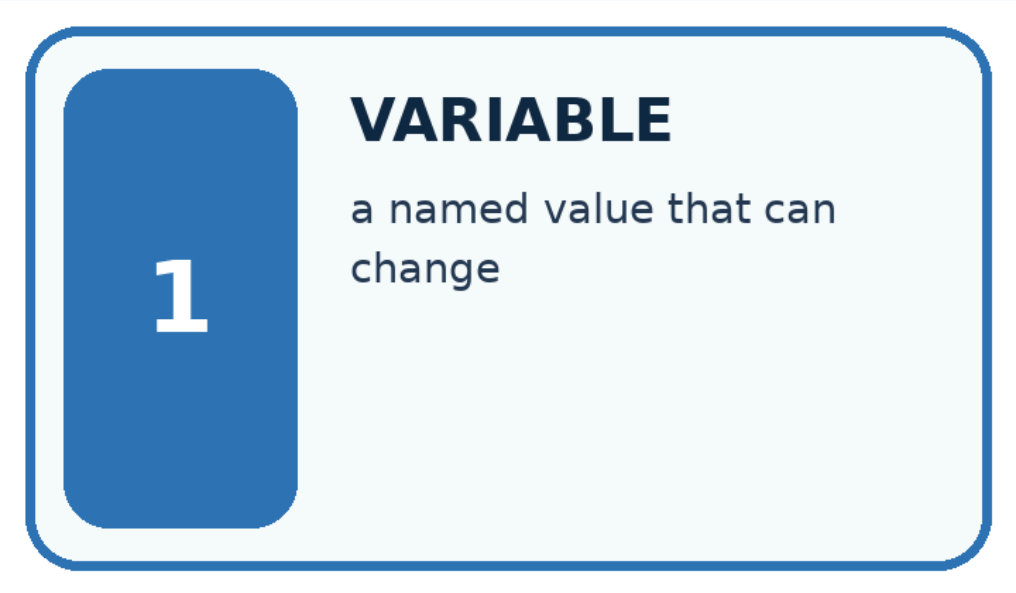
Word	Meaning
Initialization	Giving a variable its starting value
Trace table	A record of variable values as instructions execute

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

VOCABULARY LAB

Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.



</details>

2

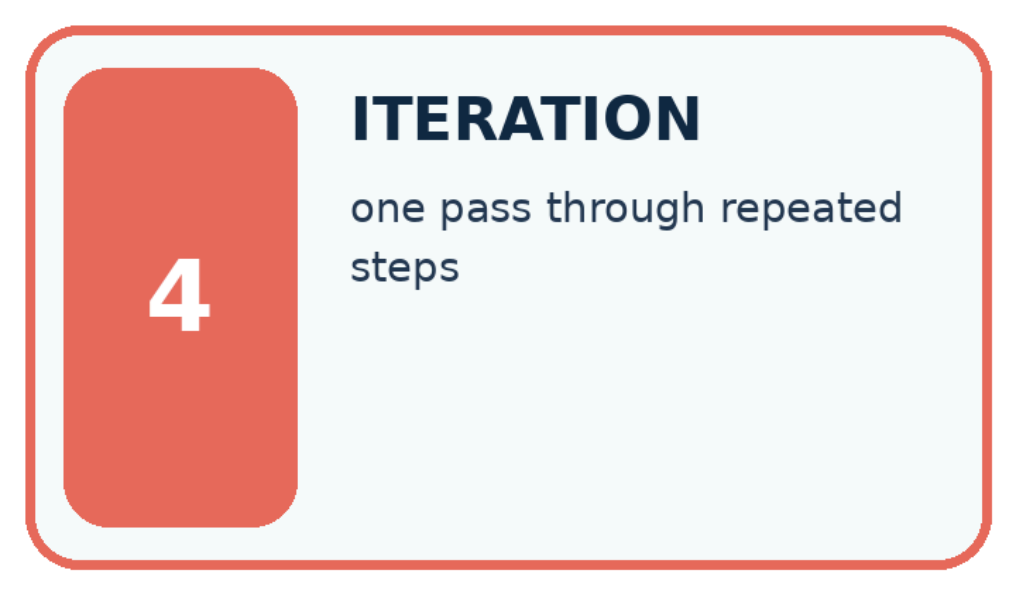
STATE

all relevant values at one moment

3

TRACE

record each step in order



Words used in the closer explanation

▼ **Choose a stronger thinking challenge**

REFERENCE · SECTION 10

Choose a stronger thinking challenge

> Choose the next useful challenge rather than treating the whole ladder as compulsory.

Pick your next step

- 1 • NAME: Define the concept without repeating the textbook.
- 2 • **TRACE**: Show the information, **state**, branch, calculation, or **evidence** path.
- 3 • APPLY: Solve a different example independently.
- 4 • CHALLENGE: Find a boundary case, counterexample, or unfair outcome.

- 5 • EXPLAIN LIMITS: Say when the result should not be trusted or used alone.

- My strongest **evidence** of independent understanding:

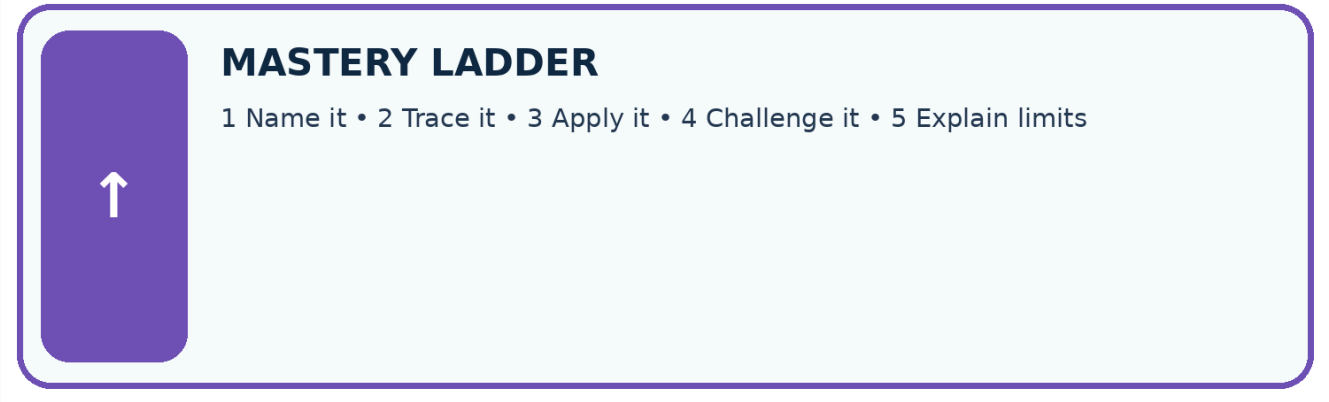
Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

MASTERY

From Knowing the Word to Owning the Idea

Move upward only when you can produce **evidence** without copying the worked example.



▼ Reference: check your explanation

Reference: check your explanation

> Use after practice. Build a repeated addition independently is the selected independent case.

Check the evidence and its limit

- 1. My fresh example, **trace**, calculation, or design:
- 2. My **evidence** that it works (include a boundary or missing-data case):
- 3. One limitation, fairness concern, or situation needing human review:

> **Write or draw your reasoning and evidence.**

> **Write or draw your reasoning and evidence.**

Self-check and record

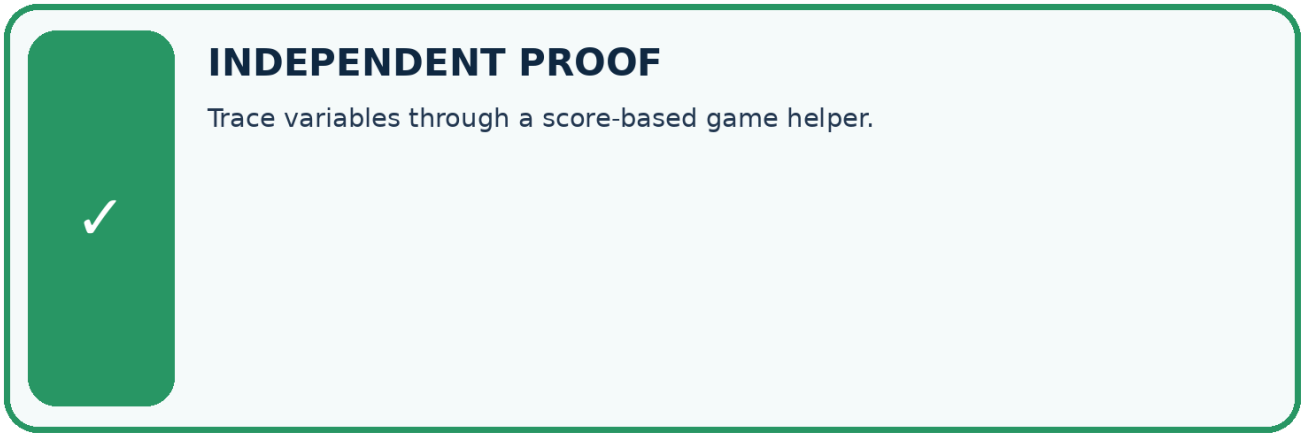
SELF-CHECK ☐ I explained the **mechanism** ☐ I used **evidence** ☐ I tested a fresh case ☐ I named a limitation ☐ I can answer 'why?'

▼ **Earlier wording, alternatives and picture transcripts**

INDEPENDENT MASTERY PROOF

Checklist for explaining this week's idea

Use after practice. Build a repeated addition independently is the selected independent case.



EXPLANATION CHECK **Trace** initialized state, range values, before/after updates and displayed output while keeping expected and actual records distinct.

Write or draw your reasoning and evidence.

▼ **Extra practice: Keep the state after each instruction**

REFERENCE · SECTION 13

Extra practice: Keep the state after each instruction

> Try the question before opening its worked explanation.

Read the supplied case and question

Set score=2; add 3; if score>4 subtract 1; double score.

Trace every value.

Your explanation

- Which detail supports the answer, and why does it matter?

Write or draw your reasoning and evidence.

▼ Check after trying

2,5,4,8. The comparison is evaluated at 5 before subtraction.

Evidence or stage	Value or result
Set score	2
Add 3	5
If greater than 4, subtract 1	4
Double	8

▼ Earlier wording, alternatives and picture transcripts

Week 4 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

The question

Worked explanation

Explain it in your own words

Trace the supplied case visually

Keep the state after each instruction

Set score=2; add 3; if score>4 subtract 1; double score.

Trace every value.

2,5,4,8. The comparison is evaluated at 5 before subtraction.

▼ Reference: original lesson checklist

REFERENCE · SECTION 15

Reference: original lesson checklist

> Use an additional checklist when it helps inspect your explanation.

Targets and evidence

By the end, I can	How I will show it
Trace variable values before and after each loop iteration	Explain the deciding step, show a trace or test, and state one limit.
Use the excluded stop value in range correctly	Explain the deciding step, show a trace or test, and state one limit.
Explain initialization and test zero one and several iterations	Explain the deciding step, show a trace or test, and state one limit.

Keep your route and first prediction

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: A fictional reading-token jar starts with one token and gains three per round.

▼ Extra practice: pictures, repair and feedback

REFERENCE · SECTION 17

Extra practice: pictures, repair and feedback

> Attempt each question before opening its explanation. Use one browser or paper copy.

First supported question

- A fictional reading-token jar starts with one token and gains three per round.
- Which labelled step matches this detail? Print 13 once, after all four updates.
- 1. **Iteration** values
- 2. Updated totals
- 3. After the **loop**
- 4. Initial **state**
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> _____
>
> _____
>
> _____
>
> _____

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: After the **loop** Print 13 once, after all four updates. The turn value labels an **iteration**; it is not the number added. The fixed update adds three each time.

Four **iteration**s take the total from 1 to 13. Keeping print outside the **loop** produces one final display rather than four intermediate displays. Initialise total before the **loop** so a zero-**iteration** run still has a defined value.

Changed-case question

- Changed-case practice: Use range(0) instead of range(4). Which conclusion is supported by the supplied facts?
- 1. The **loop** body never runs and the printed total is 1.
- 2. This one example guarantees correct results for every future **input**.
- 3. We can decide the result without examining the changed **input** or the procedure.
- Hint if needed: Use the changed condition and the deciding step in the pictorial **trace**. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check the changed case**

After your attempt, compare with the supported explanation: The **loop** body never runs and the printed total is 1. The iterable contains no turn values; the initial assignment still happened.

Repair and save your work

Review Branching Logic (ai-2/3). Goal: **Trace** the selected branch in an if elif else chain. Short practice: Give expected labels for –1,0,10,11 and identify the equality boundary for Short..

Return to this week and explain what you changed.

Review Precision in Algorithms (ai-2/1). Goal: Translate a short paper calculation into a Python script. Short practice: If print(total) moves before total=total+2, what gets printed for the original **input**?.

Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a **trace** rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work
</details>

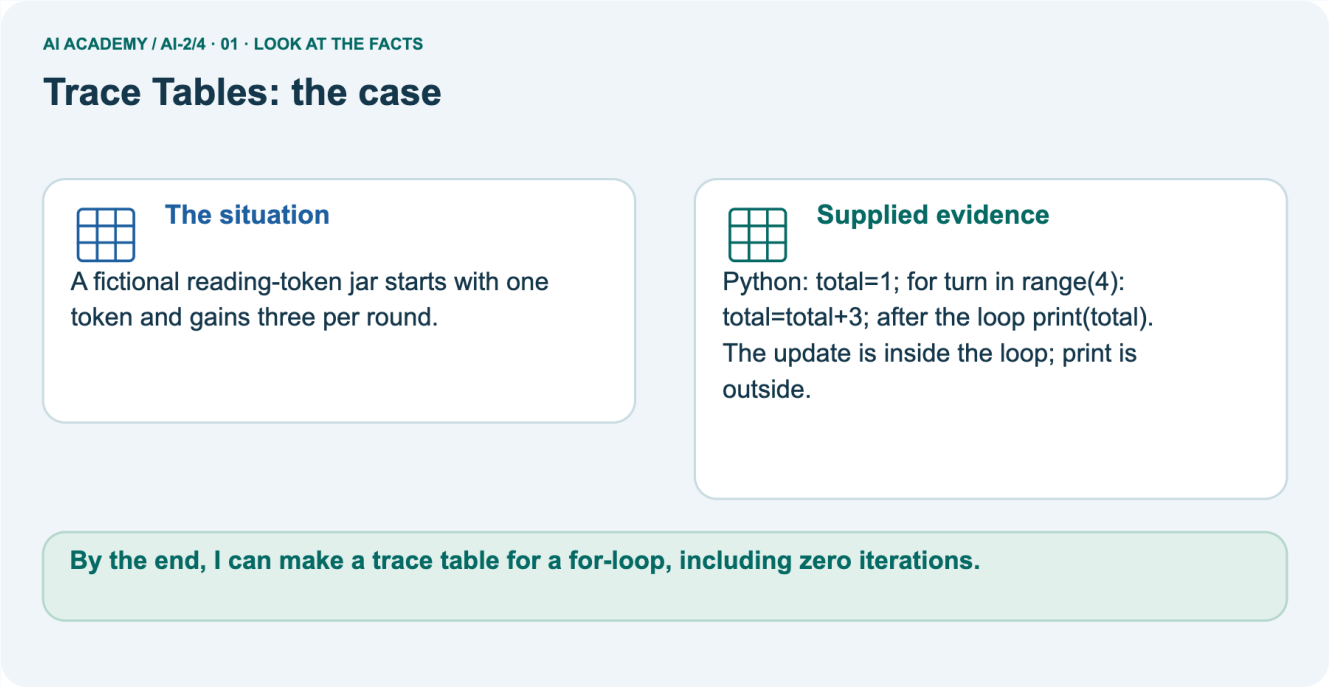
<details>
<summary>Reference: the six-picture journey</summary>
</details>

REFERENCE · SECTION 18

Reference: the six-picture journey

> Revisit the picture that serves the current question; key diagrams already sit beside the main explanation.

01. Trace Tables: the case

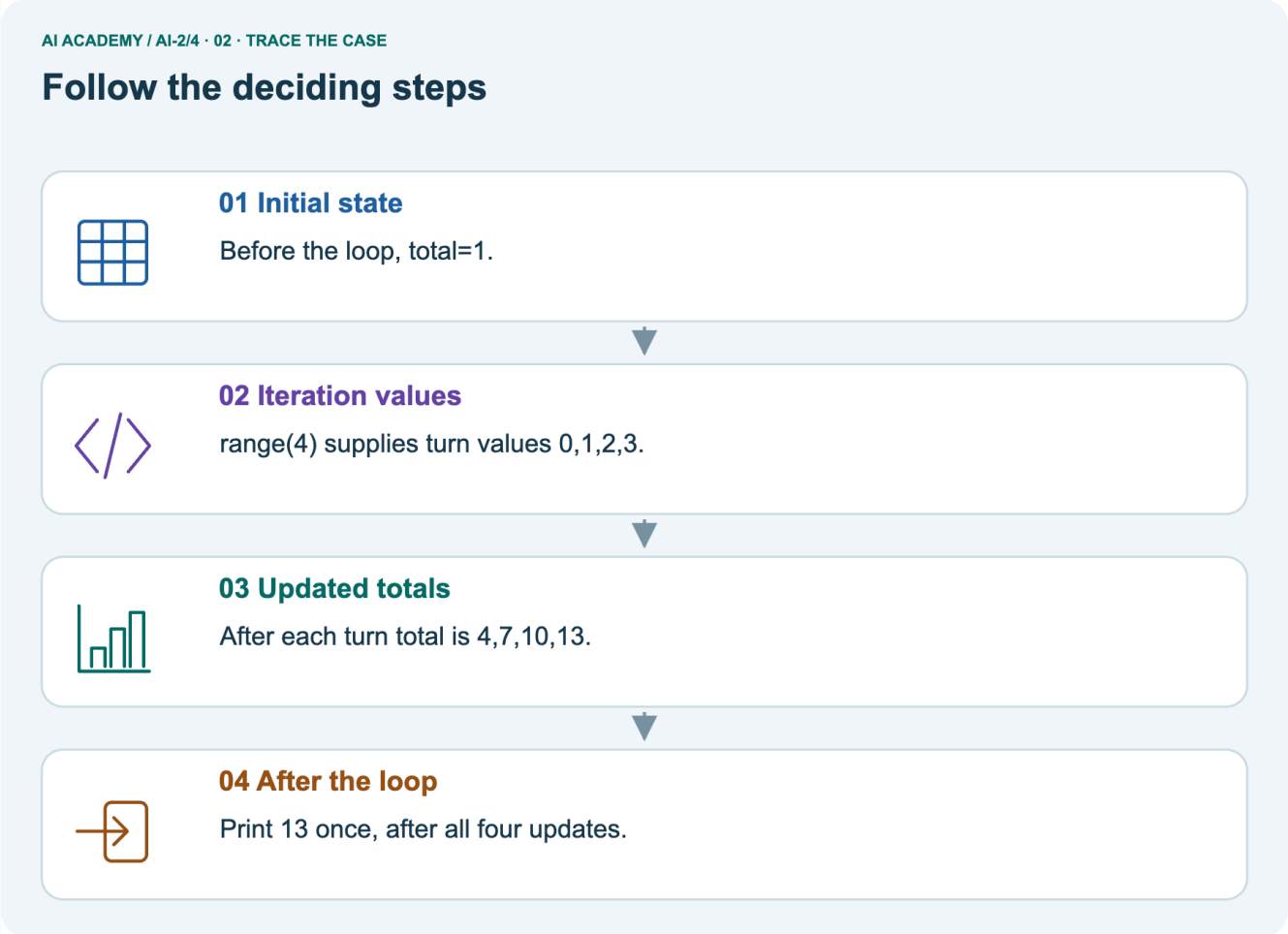


Follow the labelled picture with the matching explanation. Open the vector diagram

A fictional reading-token jar starts with one token and gains three per round. Supplied facts: Python: total=1; for turn in range(4): total=total+3; after the **loop** print(total). The update is inside the **loop**; print is outside.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Initial **state**: Before the **loop**, total=1. → **Iteration** values: range(4) supplies turn values 0,1,2,3. → Updated totals: After each turn total is 4,7,10,13. → After the **loop**: Print 13 once, after all four updates.

Point to each numbered stage and explain which recorded **input** it uses.

03. Loop totals after each iteration

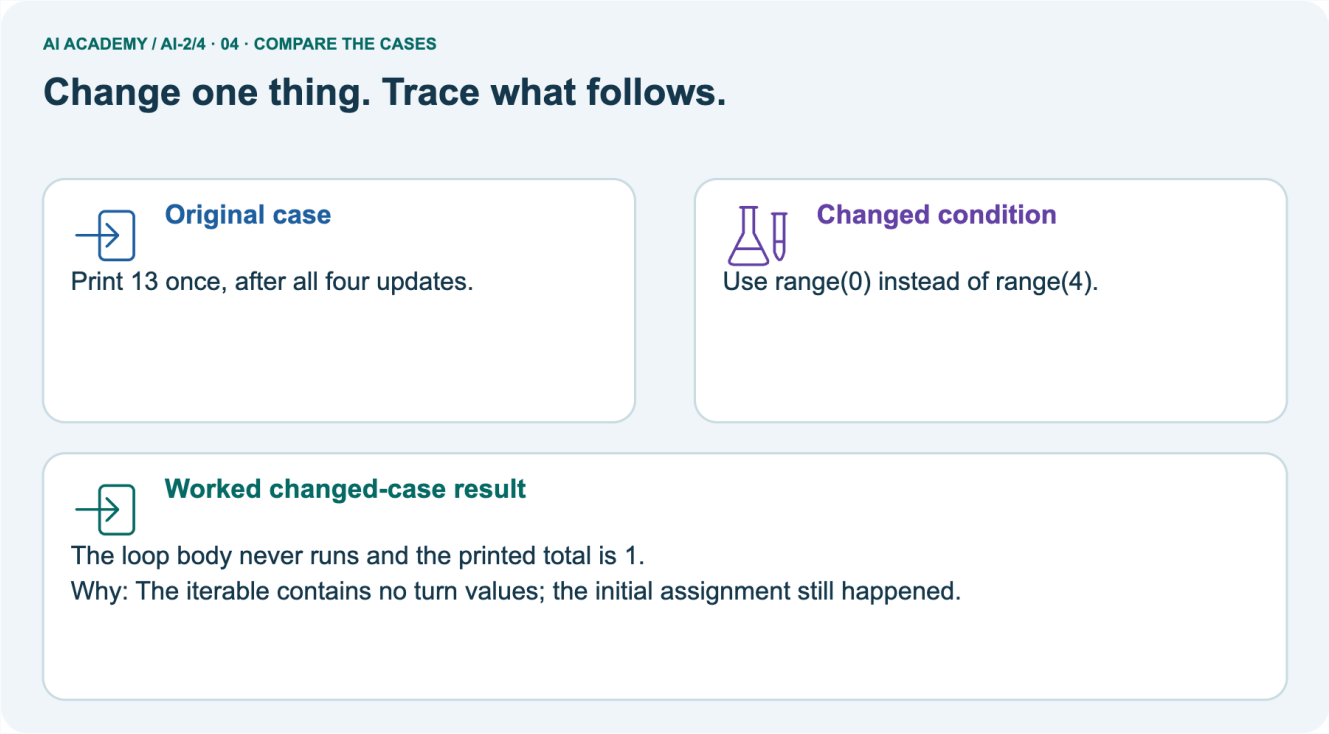


Follow the labelled picture with the matching explanation. Open the vector diagram

Loop totals after each **iteration**. First: turn = 0, total after update = 4. Second: turn = 1, total after update = 7. Third: turn = 2, total after update = 10. Fourth: turn = 3, total after update = 13.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



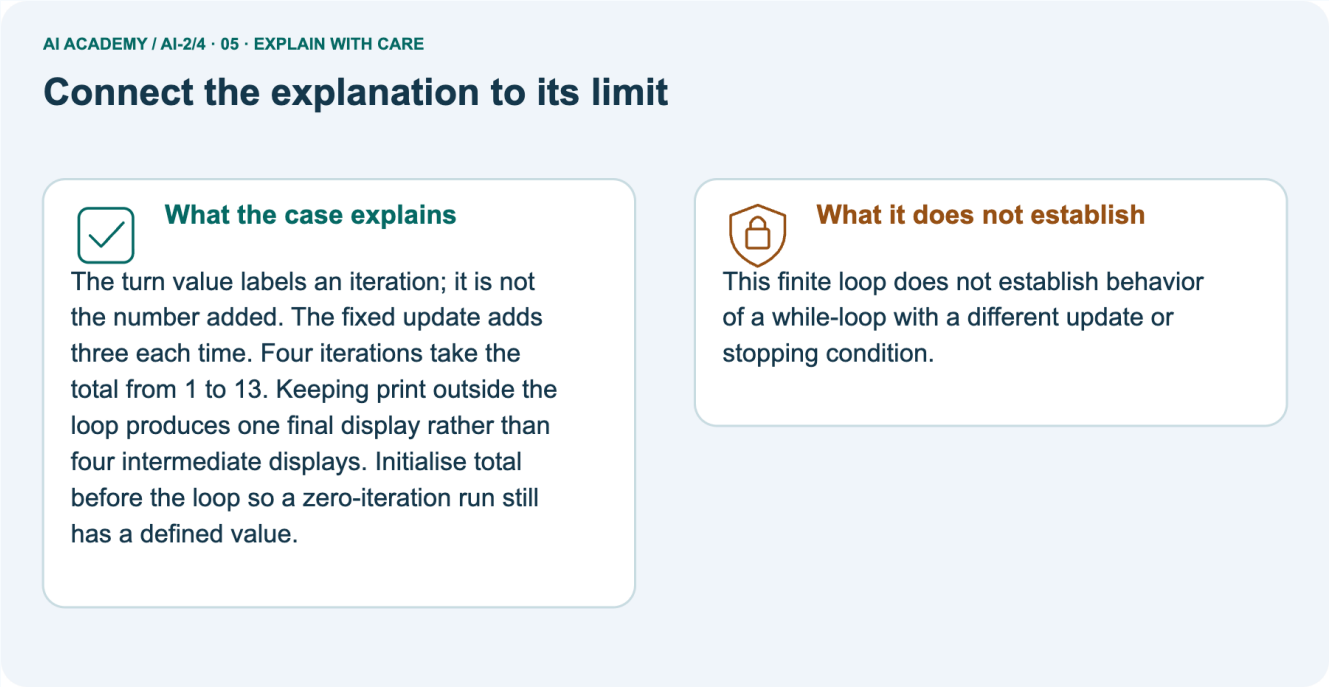
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Print 13 once, after all four updates. Changed condition: Use range(0) instead of range(4). Expected result: The **loop** body never runs and the printed total is 1.

Reason: The iterable contains no turn values; the initial assignment still happened.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

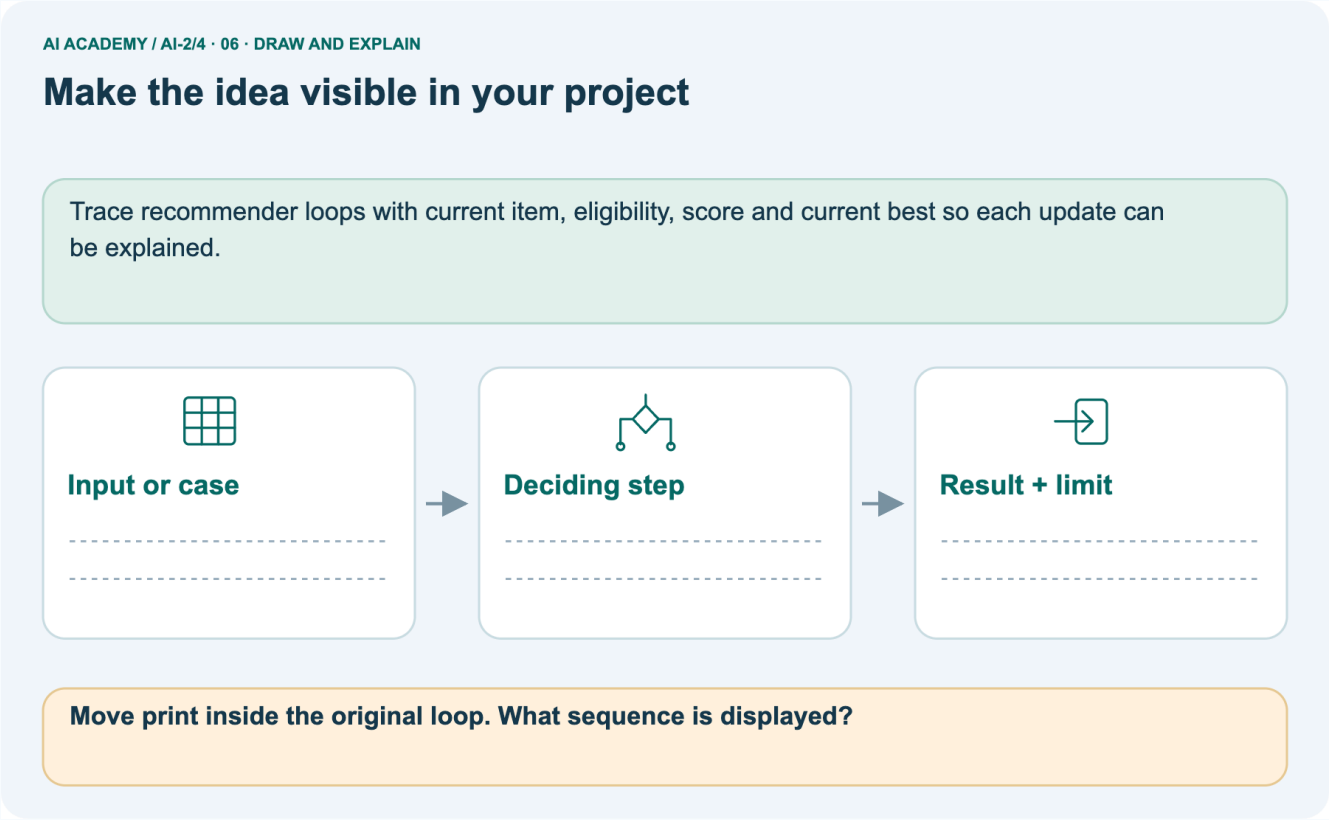
The turn value labels an **iteration**; it is not the number added. The fixed update adds three each time. Four **iteration**s take the total from 1 to 13.

Keeping print outside the **loop** produces one final display rather than four intermediate displays. Initialise total before the **loop** so a zero-**iteration** run still has a defined value.

Limit: This finite **loop** does not establish behavior of a while-**loop** with a different update or stopping condition.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

Trace recommender **loop**s with current item, eligibility, score and current best so each update can be explained. Draw your **input**, deciding step, result and limit. Supported question: Move print inside the original **loop**.

What sequence is displayed?

Draw your project **trace** and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Trace Tables: the case

02. Follow the deciding steps

03. Loop totals after each iteration

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.